



AnkritGupta007 / XV6-OS-Assignments ▾

[Code](#) [Issues](#) [Pull requests](#) 1 [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#)[Settings](#)

Assignment 1 #1

[Edit](#)[Code](#) ▾[Try the new experience](#)

Open

AnkritGupta007 wants to merge 5 commits into [main](#) from [assignment-1](#)

Conversation 0

Commits 5

Checks 0

Files changed 10

[Changes from all commits](#) ▾ [File filter](#) ▾ [Conversations](#) ▾ [Jump to](#) ▾

5 Makefile

```
77 OBJCOPY = $(TOOLPREFIX)objcopy
78 OBJDUMP = $(TOOLPREFIX)objdump
79 CFLAGS = -fno-pic -static -fno-builtin -
    fno-strict-aliasing -O2 -Wall -MD -ggdb -
    m32 -Werror -fno-omit-frame-pointer
80 CFLAGS += $(shell $(CC) -fno-stack-
    protector -E -x c /dev/null >/dev/null
    2>&1 && echo -fno-stack-protector)
81 ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
82 # FreeBSD ld wants ``elf_i386_fbsd''
181 _usertests\
182 _wc\
183 _zombie\
184
185 fs.img: mkfs README $(UPROGS)
186 ./mkfs fs.img README $(UPROGS)
250 EXTRA=\
251 mkfs.c ulib.c user.h cat.c echo.c
    forktest.c grep.c kill.c\
252 ln.c ls.c mkdir.c rm.c stressfs.c
    usertests.c wc.c zombie.c\
253 printf.c umalloc.c\
```

```
77 OBJCOPY = $(TOOLPREFIX)objcopy
78 OBJDUMP = $(TOOLPREFIX)objdump
79 CFLAGS = -fno-pic -static -fno-builtin -
    fno-strict-aliasing -O2 -Wall -MD -ggdb -
    m32 -Werror -fno-omit-frame-pointer
80 + CFLAGS += -Wno-array-bounds -Wno-infinite-
    recursion
81 CFLAGS += $(shell $(CC) -fno-stack-
    protector -E -x c /dev/null >/dev/null
    2>&1 && echo -fno-stack-protector)
82 ASFLAGS = -m32 -gdwarf-2 -Wa,-divide
83 # FreeBSD ld wants ``elf_i386_fbsd''
182 _usertests\
183 _wc\
184 _zombie\
185 + _hello\
186 + _sleep\
187
188 fs.img: mkfs README $(UPROGS)
189 ./mkfs fs.img README $(UPROGS)
253 EXTRA=\
254 mkfs.c ulib.c user.h cat.c echo.c
    forktest.c grep.c kill.c\
255 ln.c ls.c mkdir.c rm.c stressfs.c
    usertests.c wc.c zombie.c\
256 printf.c umalloc.c hello.c sleep.c\
```

```

254 -      README dot-bochsrc *.pl toc.* runoff
      runoff1 runoff.list\
255      .gdbinit.tmpl gdbutil\
256

```

```

257 +      README dot-bochsrc *.pl toc.* runoff
      runoff1 runoff.list\
258      .gdbinit.tmpl gdbutil\
259

```

▼ 19 README

... @@ -1,3 +1,20 @@

```

1 + Environment
2 + - OS: Windows Subsystem for Linux 2
   (WSL2), Ubuntu
3 +      Kernel: Linux DESKTOP-IGT6P40
   6.18.33.2-microsoft-standard-WSL2
4 + - Compiler: gcc (Ubuntu 15.2.0-16ubuntu1)
   15.2.0
5 + - Emulator: QEMU emulator version 10.2.1
   (Debian 1:10.2.1+ds-1ubuntu3.2)
6 + - xv6 source: github.com/mit-pdos/xv6-
   public (x86)
7 + - Built with: make
8 + - Run with: make qemu-nox
9 +
10 +
11 +
12 + -----
   -----
   -----
13 +
14 +
15 +
16 +
17 +

```

```

1  NOTE: we have stopped maintaining the x86
2  version of xv6, and switched
3  our efforts to the RISC-V version
   (https://github.com/mit-pdos/xv6-
   riscv.git)
48  will need to install a cross-compiler gcc
   suite capable of producing
49  x86 ELF binaries (see
   https://pdos.csail.mit.edu/6.828/).
50  Then run "make TOOLPREFIX=i386-jos-elf-".
   Now install the QEMU PC
51  - simulator and run "make qemu".

```

```

18  NOTE: we have stopped maintaining the x86
   version of xv6, and switched
19  our efforts to the RISC-V version
   (https://github.com/mit-pdos/xv6-
   riscv.git)
65  will need to install a cross-compiler gcc
   suite capable of producing
66  x86 ELF binaries (see
   https://pdos.csail.mit.edu/6.828/).
67  Then run "make TOOLPREFIX=i386-jos-elf-".
   Now install the QEMU PC
68  + simulator and run "make qemu".

```

▼ 11 hello.c

... @@ -0,0 +1,11 @@

```

1 + #include "types.h"
2 + #include "stat.h"
3 + #include "user.h"
4 +
5 + int
6 + main(int argc, char *argv[])
7 + {
8 +     printf(1, "Hello Xv6!\n");
9 +     hello();
10 +     exit();
11 + }
```

▼ ↕ 45 ls.c

```

3  #include "user.h"
4  #include "fs.h"
5
6  char*
7  - fmtname(char *path)
8  {
9  -     static char buf[DIRSIZ+1];
10
11     char *p;
12
13     // Find first character after last
14     slash.
15     for(p=path+strlen(path); p >= path && *p
16     != '/'; p--)
17         ;
18     p++;
19
20     // Return blank-padded name.
21     if(strlen(p) >= DIRSIZ)
22         return p;
23     memmove(buf, p, strlen(p));
24     memset(buf+strlen(p), ' ', DIRSIZ-
25     strlen(p));
```

```

3  #include "user.h"
4  #include "fs.h"
5
6  + int showall = 0; // set to 1 when -a is
7  + given
8  + char*
9  + fmtname(char *path, int isdir)
10  {
11  +     static char buf[DIRSIZ+2]; // +1 for
12  +     the '/', +1 for the terminator
13  +     char *p;
14  +     int len;
15
16     // Find first character after last
17     slash.
18     for(p=path+strlen(path); p >= path && *p
19     != '/'; p--)
20         ;
21     p++;
22
23     len = strlen(p);
24     if(len > DIRSIZ)
25         len = DIRSIZ;
26     memmove(buf, p, len);
27     if(isdir)
28         buf[len++] = '/'; // slash
29         goes right after the name...
30     while(len < DIRSIZ)
31         buf[len++] = ' '; // ...then
32         pad with spaces
33     buf[len] = 0;
```

```

22     return buf;
23 }
24
43
44     switch(st.type){
45     case T_FILE:
46 -     printf(1, "%s %d %d %d\n",
        fmtname(path), st.type, st.ino, st.size);
47
48         break;
49
50     case T_DIR:
51         while(read(fd, &de, sizeof(de)) ==
52 sizeof(de)){
53             if(de.inum == 0)
54                 continue;
55
56             memmove(p, de.name, DIRSIZ);
57             p[DIRSIZ] = 0;
58             if(stat(buf, &st) < 0){
59                 printf(1, "ls: cannot stat %s\n",
60 buf);
61                 continue;
62             }
63 -     printf(1, "%s %d %d %d\n",
        fmtname(buf), st.type, st.ino, st.size);
64
65         }
66 -     break;
67     }
68
69     int
70     main(int argc, char *argv[])
71     {
72 -     int i;
73
74 -     if(argc < 2){
75
76         ls(".");
77         exit();
78
79     }
80 -     for(i=1; i<argc; i++)
81
82         ls(argv[i]);

```

```

29     return buf;
30 }
31
50
51     switch(st.type){
52     case T_FILE:
53 +     printf(1, "%s %d %d %d\n",
        fmtname(path, 0), st.type, st.ino,
        st.size);
54
55         break;
56
57     case T_DIR:
58         while(read(fd, &de, sizeof(de)) ==
59 sizeof(de)){
60             if(de.inum == 0)
61                 continue;
62 +             if(!showall && de.name[0] == '.')
63 // hide dotfiles unless -a
64 +             continue;
65
66             memmove(p, de.name, DIRSIZ);
67             p[DIRSIZ] = 0;
68             if(stat(buf, &st) < 0){
69                 printf(1, "ls: cannot stat %s\n",
70 buf);
71                 continue;
72             }
73 +     printf(1, "%s %d %d %d\n",
        fmtname(buf, st.type == T_DIR), st.type,
        st.ino, st.size);
74
75         }
76         break;
77     }
78
82     int
83     main(int argc, char *argv[])
84     {
85 +     int i, npaths = 0;
86
87 +     // Pass 1: find the flag, so "ls -a dir"
88 // and "ls dir -a" both work.
89 +     for(i = 1; i < argc; i++){
90 +         if(strcmp(argv[i], "-a") == 0)
91             showall = 1;
92
93 +     // Pass 2: list every argument that
94 // isn't the flag.
95 +     for(i = 1; i < argc; i++){
96 +         if(strcmp(argv[i], "-a") == 0)
97             continue;
98         ls(argv[i]);

```

		97	+ npaths++;
		98	+ }
		99	+ if(npaths == 0) // no path given: list the current directory
		100	+ ls(".");
84	exit();	101	exit();
85	}	102	}

▼ 14 sleep.c

...

@@ -0,0 +1,14 @@

1	+ #include "types.h"
2	+ #include "stat.h"
3	+ #include "user.h"
4	+
5	+ int
6	+ main(int argc, char *argv[])
7	+ {
8	+ if(argc < 2){
9	+ printf(2, "Usage: sleep <ticks>\n");
10	+ exit();
11	+ }
12	+ sleep(atoi(argv[1]));
13	+ exit();
14	+ }

▼ 2 syscall.c

103	extern int sys_wait(void);	103	extern int sys_wait(void);
104	extern int sys_write(void);	104	extern int sys_write(void);
105	extern int sys_uptime(void);	105	extern int sys_uptime(void);
		106	+ extern int sys_hello(void);
106		107	
107	static int (*syscalls[])(void) = {	108	static int (*syscalls[])(void) = {
108	[SYS_fork] sys_fork,	109	[SYS_fork] sys_fork,
126	[SYS_link] sys_link,	127	[SYS_link] sys_link,
127	[SYS_mkdir] sys_mkdir,	128	[SYS_mkdir] sys_mkdir,
128	[SYS_close] sys_close,	129	[SYS_close] sys_close,
		130	+ [SYS_hello] sys_hello,
129	};	131	};
130		132	
131	void	133	void

▼ 1 syscall.h

20	#define SYS_link 19	20	#define SYS_link 19
21	#define SYS_mkdir 20	21	#define SYS_mkdir 20

22	#define SYS_close 21	22	#define SYS_close 21
		23	+ #define SYS_hello 22

▼ 9 sysproc.c			
89	release(&tickslock);	89	release(&tickslock);
90	return xticks;	90	return xticks;
91	}	91	}
		92	+
		93	+
		94	+ //return hello from the kernel for our custom hello message
		95	+ int
		96	+ sys_hello(void)
		97	+ {
		98	+ cprintf("Hello from the kernel!\n");
		99	+ return 0;
		100	+ }

▼ 1 user.h			
23	char* sbrk(int);	23	char* sbrk(int);
24	int sleep(int);	24	int sleep(int);
25	int uptime(void);	25	int uptime(void);
		26	+ int hello(void);
26		27	
27	// ulib.c	28	// ulib.c
28	int stat(const char*, struct stat*);	29	int stat(const char*, struct stat*);

▼ 1 usys.S			
29	SYSCALL(sbrk)	29	SYSCALL(sbrk)
30	SYSCALL(sleep)	30	SYSCALL(sleep)
31	SYSCALL(uptime)	31	SYSCALL(uptime)
		32	+ SYSCALL(hello)